

```
# Plan Projet : Application Web de Visualisation du Ciel Nocturne
## Version Optimisée - Performance Maximale & Rendu Réaliste
```

```
---
```

```
## 1. STACK TECHNOLOGIQUE FINALE (100% Gratuite & Open Source)
```

```
### 1.1 Architecture complète
```

FRONTEND |— React 18+ (UI framework) |— Vite (build tool - plus rapide que CRA) |— Three.js r160+ (moteur 3D WebGL) |— React Three Fiber (bridge React-Three.js) |— React Three Drei (helpers 3D) |— Axios (HTTP client) |— Zustand (state management léger)

BACKEND |— Python 3.11+ (langage) |— FastAPI (framework web async) |— Uvicorn (serveur ASGI) |— AstroPy (calculs astronomiques) |— Skyfield (positions précises) |— SQLAlchemy (ORM) |— Pydantic (validation données) |— Redis (cache optionnel mais recommandé)

BASE DE DONNÉES |— PostgreSQL 15+ (stockage données) |— Redis 7+ (cache en mémoire)

INFRASTRUCTURE |— Docker + Docker Compose (conteneurisation) |— Nginx (reverse proxy - production) |— Git + GitHub (versioning)

DÉPLOIEMENT (GRATUIT) |— Railway.app (backend + PostgreSQL + Redis) |— Vercel (frontend) |— GitHub Actions (CI/CD - optionnel)

```
**Tout est 100% gratuit et open source, aucune licence payante.**
```

```
---
```

```
## 2. POURQUOI POSTGRESQL ? (Explication détaillée)
```

```
### 2.1 Besoins de stockage
```

```
**Volumes de données** :
```

- ****119 000+ étoiles**** (HYG Database) : ~50 Mo de données structurées
- ****88 constellations**** : patterns (lignes) + limites (polygones) : ~5 Mo
- ****Points d'observation**** : 50-100 villes : ~10 Ko
- ****TOTAL**** : ~55 Mo de données relationnelles

```
### 2.2 Pourquoi une base de données relationnelle ?
```

```
**Alternative 1 : Fichiers JSON statiques**
```

- **✗** Pas de requêtes complexes (filtrage par magnitude, zone géographique)
- **✗** Chargement complet en mémoire (119k étoiles = 50 Mo RAM constamment)
- **✗** Pas d'indexation → recherche d'une étoile par ID = $O(n)$
- **✗** Pas de jointures (étoiles ↔ constellations)

****Alternative 2 : SQLite****

- ☒ Léger, embarqué
- ☒ Performance limitée en concurrent (write locks)
- ☒ Pas de support spatial natif (géométries constellations)
- ☒ Scalabilité faible

****☒ PostgreSQL choisi pour** :**

1. ****Indexation spatiale**** : extension PostGIS pour géométries constellations (polygones)
2. ****Requêtes complexes optimisées**** :

```
```sql
SELECT * FROM stars
WHERE magnitude < 6.0
AND dec BETWEEN -30 AND 60
ORDER BY magnitude ASC
LIMIT 5000;
```

→ Avec index B-tree sur **magnitude** et **dec** : **< 10ms** vs 500ms+ sans BD

### 3. Jointures performantes :

```
SELECT s.*, c.name as constellation_name
FROM stars s
JOIN constellation_stars cs ON s.id = cs.star_id
JOIN constellations c ON cs.constellation_id = c.id
WHERE c.abbreviation = 'ORI';
```

4. **Transactions ACID** : cohérence données (importante si ajout features futures : favoris utilisateur, etc.)
5. **Extensibilité** : PostGIS pour calculs géométriques avancés (point dans polygone pour savoir si étoile dans constellation)
6. **Performance** : connection pooling, requêtes préparées, matérialized views possibles

## 2.3 Exemple concret d'optimisation

**Sans PostgreSQL** (fichier JSON chargé en mémoire) :

```
Trouver 5000 étoiles visibles (mag < 6, altitude > 0) depuis Paris
→ Parcourir 119 000 étoiles en Python
→ Calculer position de chacune (coûteux)
→ Filtrer
→ Temps : 2-3 secondes
```

**Avec PostgreSQL :**

1. Pré-filtrage SQL : `SELECT * WHERE magnitude < 6 AND dec > -30`  
→ Retourne 8000 étoiles (indexé, < 10ms)
2. Calcul position uniquement sur 8000 étoiles (au lieu de 119k)
3. Filtrage final altitude > 0  
→ Temps : 300-500ms (gain x4-6)

## 3. INFRASTRUCTURE ET CONFIGURATION BASE DE DONNÉES

### 3.1 Setup développement (local avec Docker)

#### Structure projet :

```

night-sky-app/
├── backend/
│ ├── app/
│ │ ├── __init__.py
│ │ ├── main.py
│ │ ├── models/ (SQLAlchemy models)
│ │ ├── repositories/ (Data access)
│ │ ├── services/ (Business logic)
│ │ └── api/ (FastAPI routes)
│ ├── requirements.txt
│ ├── Dockerfile
│ └── .env
├── frontend/
│ ├── src/
│ ├── package.json
│ ├── Dockerfile
│ └── vite.config.js
├── data/ (Données initiales)
│ ├── hyg_v37.csv
│ └── constellations.json
├── docker-compose.yml
└── README.md

```

#### docker-compose.yml (environnement dev complet) :

```

version: '3.8'

services:
 # Base de données PostgreSQL
 postgres:
 image: postgres:15-alpine
 container_name: nightsky-postgres
 environment:
 POSTGRES_DB: nightsky_db
 POSTGRES_USER: nightsky_user

```

```
 POSTGRES_PASSWORD: dev_password_change_in_prod
 ports:
 - "5432:5432"
 volumes:
 - postgres_data:/var/lib/postgresql/data
 - ./data/init.sql:/docker-entrypoint-initdb.d/init.sql
 restart: unless-stopped
 healthcheck:
 test: ["CMD-SHELL", "pg_isready -U nightsky_user"]
 interval: 10s
 timeout: 5s
 retries: 5

Cache Redis
redis:
 image: redis:7-alpine
 container_name: nightsky-redis
 ports:
 - "6379:6379"
 volumes:
 - redis_data:/data
 restart: unless-stopped
 command: redis-server --appendonly yes --maxmemory 256mb --maxmemory-policy
allkeys-lru

Backend FastAPI
backend:
 build:
 context: ./backend
 dockerfile: Dockerfile
 container_name: nightsky-backend
 environment:
 DATABASE_URL:
postgres://nightsky_user:dev_password_change_in_prod@postgres:5432/nightsky_db
 REDIS_URL: redis://redis:6379/0
 ENVIRONMENT: development
 ports:
 - "8000:8000"
 volumes:
 - ./backend:/app
 depends_on:
 postgres:
 condition: service_healthy
 redis:
 condition: service_started
 restart: unless-stopped
 command: uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

Frontend React (dev avec hot reload)
frontend:
 build:
 context: ./frontend
 dockerfile: Dockerfile.dev
 container_name: nightsky-frontend
```

```

 ports:
 - "5173:5173"
 volumes:
 - ./frontend:/app
 - /app/node_modules
 environment:
 VITE_API_URL: http://localhost:8000
 depends_on:
 - backend
 command: npm run dev -- --host

volumes:
 postgres_data:
 redis_data:

```

### Avantages de cette config Docker :

- ☒ **Isolation complète** : chaque service dans son conteneur
- ☒ **Reproductible** : même environnement dev/prod
- ☒ **Un seul `docker-compose up`** : tout démarre
- ☒ **Volumes persistants** : données PostgreSQL conservées
- ☒ **Hot reload** : modifications code visibles instantanément
- ☒ **Healthchecks** : backend attend que PostgreSQL soit prêt

## 3.2 Configuration PostgreSQL optimisée pour performance

**backend/data/init.sql** (exécuté au premier démarrage) :

```

-- Extension PostGIS pour géométries (optionnel mais recommandé)
CREATE EXTENSION IF NOT EXISTS postgis;

-- Index pour performances maximales
CREATE INDEX idx_stars_magnitude ON stars(magnitude);
CREATE INDEX idx_stars_ra_dec ON stars(ra, dec);
CREATE INDEX idx_stars_hip_id ON stars(hip_id);

-- Index spatial pour constellations
CREATE INDEX idx_constellations_boundaries ON constellations USING
GIST(boundaries);

-- Configuration pour meilleures performances
ALTER SYSTEM SET shared_buffers = '256MB';
ALTER SYSTEM SET effective_cache_size = '1GB';
ALTER SYSTEM SET work_mem = '16MB';
ALTER SYSTEM SET maintenance_work_mem = '128MB';

```

## 3.3 Import données (script Python à exécuter une fois)

**backend/scripts/import\_data.py** :

```

"""
Script d'import des données astronomiques dans PostgreSQL
À exécuter une seule fois après docker-compose up

Usage:
 docker exec nightsky-backend python scripts/import_data.py
"""

import pandas as pd
from sqlalchemy import create_engine
from app.config import settings

def import_hyg_database():
 """
 Importe HYG Database v3.7 (119 000 étoiles)
 Source: https://github.com/astronexus/HYG-Database
 """
 engine = create_engine(settings.DATABASE_URL)

 # Téléchargement et lecture CSV
 df = pd.read_csv('data/hyg_v37.csv')

 # Nettoyage et transformation
 df_clean = df[['id', 'hip', 'proper', 'ra', 'dec', 'mag', 'spect',
'dist']].copy()
 df_clean.columns = ['id', 'hip_id', 'name', 'ra', 'dec', 'magnitude',
'spectral_type', 'distance_ly']

 # Import dans PostgreSQL (bulk insert ultra rapide)
 df_clean.to_sql('stars', engine, if_exists='replace', index=False,
method='multi', chunksize=5000)

 print(f"✅ {len(df_clean)} étoiles importées")

Import constellations, points d'observation, etc.
...

```

**Temps d'import** : ~30 secondes pour 119k étoiles (bulk insert PostgreSQL très optimisé)

### 3.4 Production : Railway.app (100% gratuit)

#### Pourquoi Railway ?

- ☒ **Gratuit** : 500h compute/mois + 1 Go PostgreSQL + 1 Go Redis (largement suffisant)
- ☒ **PostgreSQL inclus** : provisioning automatique
- ☒ **Redis inclus** : option à activer
- ☒ **Deploy depuis GitHub** : git push = déploiement auto
- ☒ **HTTPS gratuit** : certificats SSL auto
- ☒ **Backups automatiques** : PostgreSQL sauvegardé quotidiennement

**Alternative gratuite** : Render.com (similaire, 750h/mois gratuit)

**Setup Railway :**

1. Créer compte sur railway.app (GitHub OAuth)
2. New Project → Deploy PostgreSQL
3. New Service → Deploy from GitHub (ton repo backend)
4. Add Redis service
5. Variables d'environnement auto-configurées

**Aucun serveur perso nécessaire, tout managé par Railway.**

---

## 4. DONNÉES ET APIs (Sources finales retenues)

### 4.1 Catalogue d'étoiles : HYG Database v3.7

**Source** : <https://github.com/astronexus/HYG-Database>

- **Licence** : Open Database License (ODbL) - gratuite
- **Contenu** : 119 000 étoiles avec magnitude, positions, noms
- **Format** : CSV (téléchargement unique)
- **Usage** : Import local dans PostgreSQL
- **Avantage** : données complètes, pas d'API externe = pas de latence

### 4.2 Constellations : Stellarium + D3-Celestial

**Sources combinées :**

1. **Stellarium** (<https://github.com/Stellarium/stellarium>) - Patterns de lignes
  - Licence : GPL v2 (open source)
  - Fichier : [skycultures/western/constellationship.fab](#)
2. **D3-Celestial** (<https://github.com/ofrohn/d3-celestial>) - Limites GeoJSON
  - Licence : BSD (open source)
  - Fichiers : [data/constellations.\\*.json](#)

**Avantage** : données statiques, téléchargées une fois, aucune dépendance runtime

### 4.3 Calculs astronomiques : AstroPy + Skyfield (100% local)

**Aucune API externe** - Tout calculé côté backend Python :

- Transformations coordonnées (équatoriales → horizontales)
- Temps sidéral local
- Positions exactes au timestamp t
- Visibilité (altitude > 0°)

**Bibliothèques :**

- **AstroPy** : GPL v3 (open source)
- **Skyfield** : MIT License (open source)

## 4.4 Points d'observation : Liste statique pré-définie

**50 villes mondiales** avec coordonnées (données publiques) :

```
[
 {"id": 1, "name": "Paris", "lat": 48.8566, "lon": 2.3522, "tz": "Europe/Paris"},
 {"id": 2, "name": "Tokyo", "lat": 35.6762, "lon": 139.6503, "tz": "Asia/Tokyo"},
 // ... 48 autres
]
```

**Pas besoin d'API de géolocalisation** pour le MVP.

---

## 5. OPTIMISATIONS POUR RENDU QUALITÉ IMAGE RÉELLE

### 5.1 Globe terrestre 3D - Rendu photoréaliste

**Textures haute résolution (gratuites) :**

1. **NASA Visible Earth** (domaine public) :

- Texture jour : 8192x4096 pixels (Blue Marble Next Generation)
- URL : <https://visibleearth.nasa.gov/images/73909/december-blue-marble-next-generation-w-topography-and-bathymetry>

2. **Normal map** pour relief 3D :

- 4096x2048 pixels (bump mapping)
- Source : NASA + traitement

3. **Specular map** pour océans brillants :

- 2048x1024 pixels

4. **Night lights** (lumières villes la nuit) :

- NASA Black Marble (2048x1024)

**Configuration Three.js pour rendu réaliste :**

```
// Matériau PBR (Physically Based Rendering)
const earthMaterial = new THREE.MeshStandardMaterial({
 map: dayTexture, // 8K texture jour
 normalMap: normalTexture, // Relief
 roughnessMap: specularMap, // Océans brillants
 emissiveMap: nightTexture, // Lumières nocturnes
 emissive: new THREE.Color(0x444444),
 metalness: 0.1,
 roughness: 0.9
});
```

```
// Atmosphère (shader custom)
const atmosphereMaterial = new THREE.ShaderMaterial({
 vertexShader: atmosphereVertexShader,
 fragmentShader: atmosphereFragmentShader,
 blending: THREE.AdditiveBlending,
 side: THREE.BackSide,
 transparent: true
});
```

### Performance :

- Texture 8K compressée : ~15 Mo (chargement unique)
- LOD (Level of Detail) : texture 2K si zoom arrière
- WebP format : -30% taille vs JPEG

## 5.2 Ciel nocturne - Rendu réaliste

### Techniques avancées :

#### 1. Instanced rendering pour étoiles :

```
const starGeometry = new THREE.SphereGeometry(1, 8, 8);
const starMesh = new THREE.InstancedMesh(
 starGeometry,
 starMaterial,
 5000 // 5000 étoiles en une seule draw call
);
```

→ **Gain** : 5000 draw calls → 1 draw call = x100 performance

#### 2. Taille étoiles basée sur magnitude :

```
// Échelle logarithmique (comme œil humain)
const size = Math.pow(10, (6 - magnitude) / 2.5) * 0.5;
```

#### 3. Couleur réaliste (température stellaire) :

- Classe O/B : bleu (#9bb0ff)
- Classe A : blanc-bleu (#cad7ff)
- Classe F/G : jaune-blanc (#fff4ea)
- Classe K : orange (#ffd2a1)
- Classe M : rouge (#fcc6f)

#### 4. Effets visuels :

- Bloom post-processing (étoiles brillantes avec halo)
- HDR tone mapping
- Anti-aliasing (FXAA ou SMAA)

### 5. Voie lactée (texture NASA/ESA) :

- Panorama 360° 8192x4096
- Source : ESA Gaia mission (domaine public)
- Opacité variable selon pollution lumineuse

### Post-processing stack :

```
import { EffectComposer } from 'three/examples/jsm/postprocessing/EffectComposer';
import { UnrealBloomPass } from
'three/examples/jsm/postprocessing/UnrealBloomPass';

const composer = new EffectComposer(renderer);
composer.addPass(renderPass);
composer.addPass(new UnrealBloomPass(
 new THREE.Vector2(window.innerWidth, window.innerHeight),
 1.5, // strength
 0.4, // radius
 0.85 // threshold
));
```

## 5.3 Performance cible

### Objectifs :

- **60 FPS** constant (rotation globe + ciel)
- **< 2 secondes** chargement initial
- **< 500 ms** calcul + affichage 5000 étoiles
- **Smooth 60 FPS** transitions caméra

### Optimisations clés :

1. **Web Workers** : calculs positions étoiles en background thread
2. **Frustum culling** : étoiles hors champ non rendues
3. **Texture compression** : KTX2/Basis Universal (-70% taille)
4. **Code splitting** : lazy load modules Three.js
5. **Service Worker** : cache textures offline

---

## 6. ARCHITECTURE TECHNIQUE DÉTAILLÉE

### 6.1 Backend FastAPI - Structure POO

```
backend/app/
├── main.py # Point d'entrée FastAPI
├── config.py # Configuration (env vars)
├── database.py # SQLAlchemy engine + session
├──
└── models/ # Entités SQLAlchemy (ORM)
```

```

├── __init__.py
├── star.py # Modèle Star
├── constellation.py # Modèle Constellation
├── observation_point.py

├── schemas/ # Pydantic schemas (validation)
├── ├── __init__.py
├── ├── star.py # StarResponse, StarDetail
├── └── constellation.py # ConstellationResponse

├── repositories/ # Data Access Layer
├── ├── __init__.py
├── ├── star_repository.py
├── └── constellation_repository.py

├── services/ # Business Logic Layer
├── ├── __init__.py
├── ├── astronomy_service.py # Calcule positions
├── ├── constellation_service.py
├── └── cache_service.py # Redis wrapper

├── api/ # API Routes (endpoints)
├── ├── __init__.py
├── ├── stars.py # /api/stars/*
├── ├── constellations.py
├── └── observation_points.py

└── utils/
 ├── __init__.py
 ├── coordinate_transforms.py
 └── performance.py # Decorateurs pour monitoring

```

### Exemple classe Star (POO) :

```

models/star.py
from sqlalchemy import Column, Integer, String, Float
from app.database import Base

class Star(Base):
 """
 Entité représentant une étoile
 """
 __tablename__ = "stars"

 id = Column(Integer, primary_key=True)
 hip_id = Column(Integer, index=True) # Hipparcos ID
 name = Column(String, nullable=True)
 ra = Column(Float, nullable=False) # Ascension droite (J2000)
 dec = Column(Float, nullable=False) # Déclinaison (J2000)
 magnitude = Column(Float, index=True)
 spectral_type = Column(String)
 distance_ly = Column(Float)

```

**Exemple service (calculs) :**

```
services/astronomy_service.py
from astropy.coordinates import SkyCoord, AltAz, EarthLocation
from astropy.time import Time
import astropy.units as u

class AstronomyService:
 """
 Service de calculs astronomiques
 Responsabilité : transformations coordonnées, visibilité
 """

 @staticmethod
 def compute_horizontal_coordinates(
 ra: float,
 dec: float,
 lat: float,
 lon: float,
 timestamp: datetime
) -> dict:
 """
 Calcule azimuth et altitude d'une étoile

 @param ra: Ascension droite (degrés)
 @param dec: Déclinaison (degrés)
 @param lat: Latitude observateur (degrés)
 @param lon: Longitude observateur (degrés)
 @param timestamp: Temps d'observation (UTC)
 @return: {"azimuth": float, "altitude": float, "visible": bool}
 """

 # Position étoile (coordonnées équatoriales)
 star_coord = SkyCoord(
 ra=ra * u.degree,
 dec=dec * u.degree,
 frame='icrs'
)

 # Position observateur
 location = EarthLocation(
 lat=lat * u.degree,
 lon=lon * u.degree
)

 # Temps observation
 obs_time = Time(timestamp)

 # Transformation → coordonnées horizontales
 altaz_frame = AltAz(obstime=obs_time, location=location)
 star_altaz = star_coord.transform_to(altaz_frame)
```

```

return {
 "azimuth": star_altaz.az.degree,
 "altitude": star_altaz.alt.degree,
 "visible": star_altaz.alt.degree > 0
}

```

## 6.2 Cache Redis - Stratégie optimisée

### Quoi cacher :

#### 1. Positions étoiles calculées :

- Clé : `star_pos:{star_id}:{lat}:{lon}:{timestamp_5min}`
- TTL : 600 secondes (10 minutes)
- Gain : évite recalcul AstroPy (coûteux)

#### 2. Liste étoiles visibles par zone :

- Clé : `visible_stars:{lat}:{lon}:{mag_limit}:{timestamp_5min}`
- TTL : 600 secondes
- Gain : requête pré-calculée

#### 3. Patterns constellations :

- Clé : `constellation:{id}`
- TTL : illimité (données statiques)

### Configuration Redis pour performance max :

```

services/cache_service.py
import redis
from functools import wraps
import json

class CacheService:
 """Service de cache Redis avec compression"""

 def __init__(self):
 self.redis_client = redis.Redis(
 host='redis',
 port=6379,
 decode_responses=True,
 max_connections=50 # Pool de connexions
)

 def cached(self, ttl: int = 600):
 """
 Décorateur pour cacher résultats de fonction

 @param ttl: Durée de vie cache (secondes)
 """

```

```
def decorator(func):
 @wraps(func)
 async def wrapper(*args, **kwargs):
 # Génération clé cache
 cache_key = f"{func.__name__}:{args}:{kwargs}"

 # Tentative lecture cache
 cached_value = self.redis_client.get(cache_key)
 if cached_value:
 return json.loads(cached_value)

 # Calcul valeur
 result = await func(*args, **kwargs)

 # Stockage cache
 self.redis_client.setex(
 cache_key,
 ttl,
 json.dumps(result)
)

 return result
 return wrapper
return decorator
```

**Hit rate attendu** : 85-90% (mouvement apparent lent)

6.3 Endpoints API finaux

Endpoint	Méthode	Paramètres	Réponse	Cache
/api/observation-points	GET	-	[{id, name, lat, lon, tz}]	24h
/api/stars/visible	GET	lat, lon, timestamp, mag_limit=6	[{id, name, az, alt, mag, ...}]	10min
/api/star/{id}	GET	id	{id, name, ra, dec, mag, distance, ...}	1h
/api/constellations	GET	-	[{id, name, abbr}]	∞
/api/constellation/{id}	GET	id	{id, name, stars[], lines[]}	∞
/api/constellation/{id}/best-location	GET	id, timestamp	{point_id, visibility_score}	10min

7. PLAN DE RÉALISATION DÉTAILLÉ (12 SEMAINES)

SEMAINES 1-2 : FONDATIONS & INFRASTRUCTURE

**Objectif** : Environnement opérationnel, données importées

**Tâches Backend :**

1. ☒ Créer repos GitHub (backend + frontend séparés)
2. ☒ Setup `docker-compose.yml` (PostgreSQL + Redis + Backend)
3. ☒ Créer structure projet backend (models, services, repositories, api)
4. ☒ Configurer SQLAlchemy + Pydantic
5. ☒ Télécharger HYG Database v3.7 (CSV)
6. ☒ Écrire script `import_data.py` (import 119k étoiles)
7. ☒ Télécharger données constellations (Stellarium + D3-Celestial)
8. ☒ Importer constellations dans PostgreSQL
9. ☒ Créer modèles `Star`, `Constellation`, `ObservationPoint`
10. ☒ Test : `docker-compose up` → PostgreSQL contient données

**Tâches Frontend :**

1. ☒ Setup projet Vite + React + TypeScript
2. ☒ Installer Three.js + React Three Fiber + Drei
3. ☒ Configurer Axios (client HTTP)
4. ☒ Test : afficher un cube 3D qui tourne

**Commandes clés :**

```
Backend
cd backend
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements.txt

Frontend
cd frontend
npm install
npm run dev

Docker (tout en un)
docker-compose up -d
docker exec nightsky-backend python scripts/import_data.py
```

**Livable semaine 2 :**

- ☒ PostgreSQL avec 119k étoiles + 88 constellations
- ☒ Backend FastAPI accessible (<http://localhost:8000/docs>)
- ☒ Frontend React affichant cube 3D

---

## SEMAINES 3-4 : GLOBE 3D INTERACTIF

**Objectif** : Globe Terre photoréaliste avec points d'observation cliquables

**Tâches Backend :**

1. ☒ Endpoint `GET /api/observation-points` (retourne 50 villes)
2. ☒ Créer `ObservationPointRepository` et `ObservationPointService`
3. ☒ Tests unitaires service (PyTest)

**Tâches Frontend :**

1. ☒ Télécharger textures NASA (8K Terre jour + night lights + normal map)
2. ☒ Compresser textures en WebP
3. ☒ Créer composant `<Globe />` avec Three.js :
  - Sphère rayon 1
  - `MeshStandardMaterial` avec textures
  - Rotation auto lente (0.001 rad/frame)
  - `OrbitControls` (rotation manuelle souris)
4. ☒ Créer composant `<LocationMarker />` :
  - `SphereGeometry` petit (pins sur globe)
  - Conversion lat/lon → coordonnées 3D sphériques
  - `OnClick` event → highlight marqueur
5. ☒ Fetch `/api/observation-points` et afficher 50 marqueurs
6. ☒ Panneau latéral : afficher infos ville sélectionnée
7. ☒ Effet atmosphère (shader custom bleu clair)

**Code clé - Globe component :**

```
// frontend/src/components/Globe.jsx
import { useTexture } from '@react-three/drei';
import { useFrame } from '@react-three/fiber';
import { useRef } from 'react';

export function Globe() {
 const globeRef = useRef();

 // Chargement textures (cached par three.js)
 const [dayMap, normalMap, nightMap] = useTexture([
 '/textures/earth_day_8k.webp',
 '/textures/earth_normal_4k.webp',
 '/textures/earth_night_2k.webp'
]);

 // Rotation automatique lente
 useFrame(() => {
 globeRef.current.rotation.y += 0.001;
 });

 return (
 <mesh ref={globeRef}>
 <sphereGeometry args={[1, 64, 64]} />
 <meshStandardMaterial
 map={dayMap}
 normalMap={normalMap}
 />
 </mesh>
);
}
```

```

 emissiveMap={nightMap}
 emissive={0x222222}
 roughness={0.9}
 metalness={0.1}
 />
</mesh>
);
}

```

#### Livrable semaine 4 :

- ☒ Globe Terre photoréaliste (8K texture)
- ☒ 50 villes affichées en pins 3D
- ☒ Clic sur ville → highlight + infos affichées
- ☒ Rotation fluide 60 FPS

### SEMAINES 5-7 : CIEL NOCTURNE (CORE FEATURE)

**Objectif** : Afficher 5000+ étoiles réalistes au-dessus d'une ville

#### Tâches Backend :

1. ☒ Créer `AstronomyService.compute_horizontal_coordinates()` (AstroPy)
2. ☒ Endpoint `GET /api/stars/visible` :
  - Paramètres : lat, lon, timestamp, mag\_limit
  - Logique :
    1. Requête SQL : `SELECT * FROM stars WHERE magnitude < mag_limit AND dec BETWEEN lat-90 AND lat+90`
    2. Pour chaque étoile : calcul azimut/altitude (AstroPy)
    3. Filtrer altitude > 0
    4. Retourner JSON
3. ☒ Intégration cache Redis (décorateur `@cached`)
4. ☒ Optimisation : Web Worker Python (multiprocessing) pour calculs parallèles
5. ☒ Endpoint `GET /api/star/{id}` (détails étoile)
6. ☒ Tests : vérifier Polaris visible depuis Paris (altitude ≈ 48°)

#### Tâches Frontend :

1. ☒ Créer composant `<NightSky />` :
  - Sphère céleste inversée (rayon 100, BackSide)
  - Shader étoiles scintillantes
2. ☒ Fetch `/api/stars/visible` au clic sur ville
3. ☒ Affichage étoiles avec InstancedMesh :
  - 5000 instances en une draw call
  - Taille basée sur magnitude (échelle log)
  - Couleur basée sur classe spectrale
4. ☒ Projection fisheye (caméra FOV 120°)
5. ☒ Raycasting : détection hover/clic étoile

6. ☒ Tooltip au hover (nom étoile si connu)
7. ☒ Panneau détails au clic étoile :
  - Nom (ou HIP ID)
  - Magnitude
  - Distance (années-lumière)
  - Type spectral
  - Constellation (si applicable)
8. ☒ Post-processing : Bloom pour étoiles brillantes
9. ☒ Loading state (spinner pendant calculs)
10. ☒ Transition caméra animée (globe → ciel)

### Code clé - Calcul positions (backend) :

```
services/astronomy_service.py
from astropy.coordinates import SkyCoord, AltAz, EarthLocation
from astropy.time import Time
import astropy.units as u
from typing import List, Dict
from app.models import Star

class AstronomyService:

 @staticmethod
 async def get_visible_stars(
 lat: float,
 lon: float,
 timestamp: datetime,
 mag_limit: float = 6.0
) -> List[Dict]:
 """
 Retourne étoiles visibles depuis un point

 @param lat: Latitude observateur (degrés)
 @param lon: Longitude observateur (degrés)
 @param timestamp: Temps UTC
 @param mag_limit: Magnitude limite (défaut 6 = œil nu)
 @return: Liste [{star_id, name, azimuth, altitude, magnitude, ...}]
 """
 # Pré-filtrage SQL (rapide)
 stars = await StarRepository.get_by_magnitude_and_declination(
 mag_limit=mag_limit,
 dec_min=lat - 90,
 dec_max=lat + 90
)

 # Position observateur
 location = EarthLocation(lat=lat*u.deg, lon=lon*u.deg)
 obs_time = Time(timestamp)
 altaz_frame = AltAz(obstime=obs_time, location=location)

 visible_stars = []
```

```

 for star in stars:
 # Coordonnées équatoriales → horizontales
 star_coord = SkyCoord(ra=star.ra*u.deg, dec=star.dec*u.deg,
frame='icrs')
 star_altaz = star_coord.transform_to(altaz_frame)

 # Filtrage altitude
 if star_altaz.alt.degree > 0:
 visible_stars.append({
 'id': star.id,
 'name': star.name or f'HIP {star.hip_id}',
 'azimuth': star_altaz.az.degree,
 'altitude': star_altaz.alt.degree,
 'magnitude': star.magnitude,
 'spectral_type': star.spectral_type,
 'distance_ly': star.distance_ly
 })

 return visible_stars

```

### Code clé - Rendu étoiles (frontend) :

```

// frontend/src/components/NightSky.jsx
import { useRef, useMemo } from 'react';
import { useFrame } from '@react-three/fiber';
import * as THREE from 'three';

export function NightSky({ stars }) {
 const meshRef = useRef();

 // Conversion données étoiles → positions 3D + couleurs
 const { positions, colors, sizes } = useMemo(() => {
 const positions = [];
 const colors = [];
 const sizes = [];

 stars.forEach(star => {
 // Conversion azimuth/altitude → vecteur 3D
 const phi = (90 - star.altitude) * Math.PI / 180;
 const theta = star.azimuth * Math.PI / 180;
 const radius = 50; // Rayon sphère céleste

 const x = radius * Math.sin(phi) * Math.cos(theta);
 const y = radius * Math.cos(phi);
 const z = radius * Math.sin(phi) * Math.sin(theta);

 positions.push(x, y, z);

 // Couleur selon type spectral
 const color = getStarColor(star.spectral_type);
 colors.push(color.r, color.g, color.b);

```

```

 // Taille selon magnitude (échelle log)
 const size = Math.pow(10, (6 - star.magnitude) / 2.5) * 0.5;
 sizes.push(size);
 });

 return {
 positions: new Float32Array(positions),
 colors: new Float32Array(colors),
 sizes: new Float32Array(sizes)
 };
}, [stars]);

return (
 <points ref={meshRef}>
 <bufferGeometry>
 <bufferAttribute
 attach="attributes-position"
 count={positions.length / 3}
 array={positions}
 itemSize={3}
 />
 <bufferAttribute
 attach="attributes-color"
 count={colors.length / 3}
 array={colors}
 itemSize={3}
 />
 <bufferAttribute
 attach="attributes-size"
 count={sizes.length}
 array={sizes}
 itemSize={1}
 />
 </bufferGeometry>
 <pointsMaterial
 size={1}
 vertexColors
 sizeAttenuation
 transparent
 opacity={0.9}
 blending={THREE.AdditiveBlending}
 />
 </points>
);
}

// Mapping type spectral → couleur
function getStarColor(spectralType) {
 if (!spectralType) return new THREE.Color(0xffffffff);

 const type = spectralType.charAt(0);
 const colorMap = {
 'O': 0x9bb0ff, // Bleu
 'B': 0xaabfff, // Bleu-blanc
 };

```

```

 'A': 0xcad7ff, // Blanc
 'F': 0xf8f7ff, // Blanc jaunâtre
 'G': 0xffff4ea, // Jaune (Soleil)
 'K': 0xffd2a1, // Orange
 'M': 0xffcc6f // Rouge
 };

 return new THREE.Color(colorMap[type] || 0xffffffff);
}

```

### Performance attendue :

- Calcul 5000 étoiles : 300-500ms (avec cache : 50ms)
- Rendu 60 FPS constant
- Mémoire GPU : ~20 Mo (instancing)

### Livrable semaine 7 :

- ☒ Ciel nocturne avec 5000+ étoiles
- ☒ Couleurs réalistes (types spectraux)
- ☒ Clic étoile → détails complets
- ☒ Bloom post-processing (étoiles brillent)
- ☒ Cache Redis opérationnel

## SEMAINES 8-9 : CONSTELLATIONS

**Objectif** : Sélection constellation → navigation automatique + surbrillance

### Tâches Backend :

1. ☒ Endpoint `GET /api/constellations` (liste 88 constellations)
2. ☒ Endpoint `GET /api/constellation/{id}` :
  - Retourne : nom, étoiles formant pattern, lignes connexions
3. ☒ Service `ConstellationService.find_best_observation_point()` :
  - Logique : pour chaque point, calculer altitude centre constellation
  - Retourner point où altitude maximale
4. ☒ Endpoint `GET /api/constellation/{id}/best-location?timestamp=...`
5. ☒ Tests : Orion visible hémisphère Nord en hiver

### Tâches Frontend :

1. ☒ Panneau latéral : liste 88 constellations (search bar)
2. ☒ Clic constellation :
  - Fetch `/api/constellation/{id}/best-location`
  - Animation caméra vers point optimal
  - Fetch étoiles visibles
  - Afficher ciel avec constellation surbrillancée
3. ☒ Composant `<ConstellationPattern />` :
  - Dessiner lignes entre étoiles (Line3 Three.js)

- Couleur vive (cyan/jaune)
- Étoiles pattern agrandies
- 4. ☒ Transition fluide (tween.js ou GSAP)
- 5. ☒ Highlight persistant jusqu'à désélection

### Code clé - Tracé constellation :

```
// frontend/src/components/ConstellationPattern.jsx
import { Line } from '@react-three/drei';

export function ConstellationPattern({ constellation, starsData }) {
 return (
 <group>
 {/* Lignes entre étoiles */}
 {constellation.lines.map((line, idx) => {
 const star1 = starsData.find(s => s.id === line[0]);
 const star2 = starsData.find(s => s.id === line[1]);

 if (!star1 || !star2) return null;

 const pos1 = azAltToVector3(star1.azimuth, star1.altitude);
 const pos2 = azAltToVector3(star2.azimuth, star2.altitude);

 return (
 <Line
 key={idx}
 points={[pos1, pos2]}
 color="cyan"
 lineWidth={2}
 opacity={0.8}
 transparent
 />
);
 })}
 </group>
);
}
```

### Livrable semaine 9 :

- ☒ 88 constellations sélectionnables
- ☒ Navigation automatique vers meilleur point
- ☒ Surbrillance pattern (lignes + étoiles)
- ☒ Transitions animées fluides

---

## SEMAINES 10-11 : POLISH & OPTIMISATIONS

**Objectif** : Qualité production, UX/UI soignée

**Tâches** :

1. ☒ **Dark theme complet** (noir spatial + accents cyan/violet)
2. ☒ **Animations** :
  - Transition globe → ciel (3 secondes smooth)
  - Fade-in étoiles progressif
  - Hover states sur boutons
3. ☒ **Loading states** :
  - Skeleton loader globe
  - Progress bar calcul étoiles
  - Shimmer effect panneaux
4. ☒ **Responsive design** :
  - Mobile : panneau latéral devient bottom sheet
  - Tablet : layout adapté
  - Touch gestures (pinch zoom, swipe)
5. ☒ **Performance** :
  - Code splitting (lazy load Three.js)
  - Image lazy loading
  - Service Worker (cache textures)
  - Compression Brotli (backend)
6. ☒ **Gestion erreurs** :
  - Toast notifications (react-hot-toast)
  - Fallback si API down
  - Retry logic Axios
7. ☒ **Accessibilité** :
  - ARIA labels
  - Keyboard navigation
  - Focus indicators
8. ☒ **Documentation** :
  - Docstrings complètes (backend)
  - JSDoc (frontend)
  - README installation
  - ARCHITECTURE.md
9. ☒ **Tests** :
  - Tests unitaires backend (coverage 70%+)
  - Tests composants React (Jest)
  - Tests E2E Playwright (5 scénarios)
10. ☒ **Refactoring** :
  - Respect SOLID
  - DRY (extraction utilitaires)
  - Type strict (TypeScript + Pydantic)

#### Ajouts visuels (bonus) :

- ☒ Voie lactée texture ESA
- ☒ Grille équatoriale (optionnelle, toggle)
- ☒ Noms constellations affichés au centre pattern
- ☒ Compteur étoiles visibles en temps réel

**Livrable semaine 11 :**

- ☒ Application polie, fluide, professionnelle
  - ☒ Tests E2E passent (CI/CD green)
  - ☒ Documentation complète
  - ☒ Performance Lighthouse > 85
- 

**SEMAINE 12 : DÉPLOIEMENT & PRÉSENTATION**

**Objectif :** Mise en production, démo prête

**Tâches Déploiement :**

1. ☒ **Dockerisation finale :**
  - Multi-stage build (optimisation taille images)
  - Docker Compose production
2. ☒ **Déploiement backend Railway :**
  - Créer projet Railway
  - Provision PostgreSQL + Redis
  - Deploy depuis GitHub (auto)
  - Variables environnement
3. ☒ **Déploiement frontend Vercel :**
  - Connecter repo GitHub
  - Build automatique (Vite)
  - Config domaine personnalisé (optionnel)
4. ☒ **Configuration CORS** (backend autorise domaine Vercel)
5. ☒ **Tests production :**
  - Vérifier API accessible
  - Tester latence (< 500ms)
  - Load testing (Apache Bench, 100 req/s)
6. ☒ **Monitoring** (optionnel) :
  - Sentry (erreurs frontend)
  - Logs Railway

**Tâches Présentation :**

1. ☒ **Vidéo démo** (2-3 minutes) :
  - Intro projet
  - Rotation globe
  - Sélection ville → ciel étoilé
  - Sélection constellation
  - Détails étoile
  - Outro
2. ☒ **Slides soutenance** (15-20 slides) :
  - Problématique
  - Architecture technique
  - Stack technologique
  - Défis relevés (calculs astro, performances 3D)

- Démo live
- Perspectives évolution
- 3. ☒ **README final** :
  - Installation locale
  - Déploiement
  - API documentation
  - Credits (NASA, Stellarium, etc.)
- 4. ☒ **Rehearsal soutenance** (timing 10-15 min)

#### Livrable semaine 12 :

- ☒ Application en ligne : <https://nightsky-app.vercel.app>
  - ☒ API backend : <https://nightsky-api.railway.app>
  - ☒ Vidéo démo uploadée
  - ☒ Slides finalisées
  - ☒ Repo GitHub public avec README
- 

## 8. RÉCAPITULATIF STACK FINALE

### Technologies Backend

- **Langage** : Python 3.11
- **Framework** : FastAPI 0.109+
- **Serveur** : Uvicorn (ASGI)
- **Base données** : PostgreSQL 15
- **Cache** : Redis 7
- **ORM** : SQLAlchemy 2.0
- **Validation** : Pydantic v2
- **Calculs astro** : AstroPy 6.0 + Skyfield 1.48
- **Tests** : PyTest + Coverage

### Technologies Frontend

- **Framework** : React 18
- **Build** : Vite 5
- **Langage** : TypeScript
- **3D Engine** : Three.js r160
- **React 3D** : React Three Fiber + Drei
- **HTTP** : Axios
- **State** : Zustand
- **Styling** : Tailwind CSS
- **Tests** : Jest + React Testing Library + Playwright

### Infrastructure

- **Conteneurs** : Docker + Docker Compose
- **Dev DB** : PostgreSQL container
- **Prod Backend** : Railway.app (gratuit)

- **Prod Frontend** : Vercel (gratuit)
- **CI/CD** : GitHub Actions (optionnel)

## Données

- **Étoiles** : HYG Database v3.7 (119k étoiles, ODbL)
- **Constellations** : Stellarium + D3-Celestial (GPL/BSD)
- **Textures** : NASA Visible Earth (domaine public)
- **Voie lactée** : ESA Gaia (CC BY-SA 4.0)

**TOTAL COÛT** : 0€ (100% gratuit en dev ET production)

---

## 9. COMMANDES ESSENTIELLES

### Installation initiale (semaine 1)

```
Clone repos
git clone <ton-repo-backend>
git clone <ton-repo-frontend>

Setup backend
cd backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt

Setup frontend
cd frontend
npm install

Lancer avec Docker (recommandé)
docker-compose up -d

Import données (une seule fois)
docker exec nightsky-backend python scripts/import_data.py

Vérifications
curl http://localhost:8000/docs # Swagger API
curl http://localhost:5173 # Frontend
```

### Développement quotidien

```
Démarrer tout
docker-compose up

Logs
docker-compose logs -f backend
docker-compose logs -f postgres
```

```
Arrêter
docker-compose down

Reset complet (attention : supprime données)
docker-compose down -v
```

## Tests

```
Backend
cd backend
pytest --cov=app tests/

Frontend
cd frontend
npm run test
npm run test:e2e # Playwright
```

## Déploiement

```
Build production
docker build -t nightsky-backend ./backend
docker build -t nightsky-frontend ./frontend

Push Railway (auto depuis GitHub)
git push origin main

Build frontend Vercel (auto)
git push origin main
```

---

# 10. MÉTRIQUES DE SUCCÈS

### Performance :

- ☒ 60 FPS rotation globe + ciel
- ☒ < 2s chargement initial
- ☒ < 500ms affichage 5000 étoiles
- ☒ Lighthouse score > 85

### Fonctionnel :

- ☒ 119 000 étoiles en base
- ☒ 5000+ étoiles visibles simultanément
- ☒ 88 constellations fonctionnelles
- ☒ 50 points d'observation

**Qualité code :**

- ☒ Coverage tests > 70% backend
- ☒ 0 erreurs TypeScript
- ☒ Architecture POO respectée
- ☒ Documentation complète

**Déploiement :**

- ☒ App accessible publiquement
- ☒ Uptime > 99%
- ☒ API response time < 200ms

---

**Ce plan détaillé te donne une marche à suivre semaine par semaine. Focus sur le MVP (semaines 1-9), puis polish (10-11), puis déploiement (12). Tout est gratuit, open source, et optimisé pour performances maximales. Bon code ! 🚀**